

Gaussian Splatting SLAM Based on Loop Closure Pose Optimization and Comprehensive Loss Function

Dezhi Wang and Xinzhao Wu and Liwei Zhang* and Degui Tu

Abstract— With their high-fidelity scene representation capabilities, Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) have deeply attracted attention in Simultaneous Localization and Mapping (SLAM) community. This paper proposes a 3DGS-based SLAM method that integrates the loop closure detection module and designs a comprehensive loss function. Graphical optimizations are used to optimize trace threads and correct for accumulated errors. The experimental results show that the average improvement is about 54% compared with SplaTAM in terms of tracking, and the image similarity is also improved in terms of mapping.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) technology enables real-time mapping and localization in unknown environments. It's crucial for autonomous systems, robotics and AR/VR applications. Recent SLAM research focuses on advanced scene representation methods, including point clouds, surfels, meshes, and voxels [6][7]. The choice of mapping technique significantly impacts system design and capabilities.

In dense visual SLAM [4], traditional map-centric methods struggle to achieve high-fidelity representations and reconstruct detailed textures and repetitive scenes. Recently, Neural Radiance Fields (NeRF) [1] have gained significant attention in the SLAM field. NeRF represents a novel view synthesis method using implicit scene representation, offering photorealistic results with minimal memory requirements. With the evolving NeRF, numerous NeRF-based dense SLAM methods[2][8][9][10] have emerged rapidly.

3D Gaussian Splatting (3DGS) [3], as a new scene representation, explicitly represents scenes using 3D Gaussians as primitives. It achieves rendering quality comparable to NeRF while offering faster optimization and rendering speeds. Consequently, 3DGS is highly suitable as a map representation method in SLAM systems [11][12][13], as demonstrated by recently proposed approaches. However, current 3DGS-SLAM systems lack robust loop detection, which is crucial for correcting global drift errors and preventing map loss. This drawback becomes particularly evident in large scenes due to cumulative drift.

This paper proposes a novel dense SLAM method based on 3DGS. The method integrates ORB feature extraction [5] and FLANN[14][15][18][19] feature matching for loop closure detection and employs g2o [16][17][21][22] graph

optimization for pose optimization when a loop closure is detected. In the tracking and mapping threads, a new comprehensive loss function is designed to simultaneously optimize keyframe poses. Extensive evaluation on indoor RGB-D datasets demonstrates our method's state-of-the-art performance in tracking, rendering, and mapping. In summary, our main contributions are as follows:

- A global loop closure detection and correction method was added to the 3DGS-SLAM process. This method uses ORB feature extraction and FLANN feature matching to match the poses of model cameras. When a loop closure is detected, g2o graph optimization is employed for pose optimization, enabling real-time correction of accumulated errors in the system.
- We have designed a comprehensive loss function. In addition to depth loss and RGB loss, Multi-layer Structural Similarity Index (SSIM) loss and perceptual loss based on Learned Perceptual Image Patch Similarity (LPIPS) were added to address rendering quality.
- This method achieves an average ATE of 0.28cm and 1.49cm in tracking on the Replica and TUM-RGBD datasets. In mapping, it achieves average SSIM and LPIPS values of 0.987 and 0.051, respectively, outperforming the existing 3DGS-SLAM method.

II. RELATED WORK

A. Traditional approaches to visual SLAM

In recent years, open-source libraries in the fields of SLAM and computer vision have continued to evolve, bringing new breakthroughs to research and applications. Ila et al. proposed SLAM++ [17], an incremental solver based on g2o that efficiently handles large-scale closed-scene SLAM problems. In city-scale large-area mapping, Ramezani et al. implemented a robust laser SLAM system [18] using an improved version of g2o. Loop closure detection in ORB-SLAM2 [19] is a key component of the system, playing an important role in improving the accuracy and robustness of the SLAM system. This improvement has shown excellent performance in large-scale point cloud registration, as demonstrated by the work of Chen X [20], which showcased loop closure's superior performance in handling large-scale indoor point clouds. Moreover, the integration of these libraries with deep learning has become a new trend.

Dezhi Wang, Xinzhao Wu and Liwei Zhang are with School of Mechanical Engineering and Automation, Fuzhou University, Fuzhou, China. Degui Tu is with Fujian Yongyue Intelligent Technology Co., Ltd.

* Corresponding Author: Liwei Zhang (email: lw.zhang@fzu.edu.cn).

B. NeRF-based SLAM

In the field of RGB-D SLAM, iMAP [23] pioneered the introduction of implicit neural representations, using Multi-Layer Perceptrons (MLP) [26] for scene representation, an approach that has since gained widespread recognition and application. NICE-SLAM [24] further improved this method by adopting a hierarchical strategy, using multiple MLPs to encode scene geometry into coarse, medium, and fine-level voxel grids. To further improve mapping performance, Co-SLAM [25] and Point-SLAM [10] introduced joint coordinate encoding and parameterization methods and neural point cloud representations, respectively. ESLAM [27] took a different approach, using large-scale axis-aligned Truncated Signed Distance Functions (TSDF) as scene geometry representations, including three coarse feature planes and three fine feature planes. This method substantially reduced memory usage while significantly increasing frame rates.

In RGB SLAM, due to the lack of depth information, researchers have adopted different strategies. Go-SLAM [9] supports multi-modal inputs from monocular, stereo, and RGB-D cameras. These advancements demonstrate the rapid development of SLAM technology under deep learning applications, with different methods having their own characteristics but also their own drawbacks. These neural implicit methods typically lack online loop closure correction. When using MLPs, local over-smoothing often leads to the loss of high-frequency details.

C. 3DGS-based SLAM

Research in this field is developing rapidly, but related algorithms are relatively few. In addition to 3DGS's differentiable optimization and fast re-rendering, GS-SLAM [28] uses adaptive expansion strategies and coarse-fine camera tracking strategies to achieve high-fidelity and fast high-resolution rendering performance. MonoGS [13] performs camera tracking through direct optimization of 3D Gaussian ellipsoids, combined with geometric verification and regularization techniques, achieving continuous high-fidelity 3D scene reconstruction of small and transparent objects. Photo-SLAM [11], based on the classic ORB-SLAM3 method, integrates explicit geometric features and implicit texture representations into a super-primitive map. SplatAM [12] adopts a customized online tracking and mapping system, combined with silhouette masks, to achieve fast rendering, dense optimization, efficient map reconstruction, and structured map expansion. Gaussian-SLAM [30] introduces several new techniques such as differential depth rendering, explicit Gaussian gradients, and frame-to-model alignment, significantly enhancing its performance. However, recent methods still show slightly inferior performance in terms of frame processing rate and memory usage. GS-ICP-SLAM [29] takes a different approach, greatly improving frame processing rates by utilizing point cloud matching.

The aforementioned methods employ new technologies such as 3D Gaussian distributions and implicit neural representations, improving the accuracy and efficiency of scene representation and enabling high-fidelity 3D scene reconstruction. However, some of these methods may lead

to high memory usage and generate significant cumulative errors.

To address these issues, we can combine the expression of 3DGS with traditional SLAM loop closure detection systems, and develop methods that automatically adjust parameters and processing strategies based on scene complexity. This approach aims to achieve real-time correction of accumulated errors, improve the accuracy of 3DGS mapping, enhance image similarity, and capture more semantic information and details.

III. LOOP CLOSURE DETECTION AND COMPOSITE LOSS METHOD

A. 3DGS Scene Representation

Our proposed SLAM method utilizes 3D Gaussian representations to capture scenes, capturing both the geometric shape and appearance of the scene, thereby producing detailed dense maps and high-quality novel view synthesis. To achieve this, we will use a set of anisotropic Gaussians G to map the scene.

$$G = \{G_i : (X_i, \Sigma_i, O_i, SH_i) | i=1, \dots, N\} \quad (1)$$

Each 3D Gaussian G_i contains a position $X_i \in \mathbb{R}^3$, a covariance matrix $\Sigma_i \in \mathbb{R}^3$, an opacity $O_i \in [0,1]$, and spherical harmonics $SH_i \in \mathbb{R}^{12}$. The covariance matrix Σ_i can be simplified to the following formula:

$$\Sigma = RSS^T R^T \quad (2)$$

Where $S \in \mathbb{R}^3$ is a 3D scale vector, $R \in \mathbb{R}^{3 \times 3}$ is a rotation matrix, and the rotation matrix is represented by a quaternion q .

Differentiable Rendering: In the 3DGS scene representation, three-dimensional scenes are represented and rendered using Gaussian distributions. Specifically, the following formulas are used to calculate the color and depth value of each pixel, thereby generating the final image.

The specific roles of color and depth rendering formulas in the 3DGS scene representation are to calculate the color value $c(p)$ and depth value $d(p)$ for each pixel $p = (u, v)$ in the camera coordinate system. By performing a weighted sum of the color values and depth values of all Gaussian distributions, excluding the influence of other Gaussians, it ensures that the contribution of each Gaussian is independent. This results in the final color and depth values:

$$c(p) = \sum_i^N c_i \cdot w_i(p) \cdot \prod_j^{i-1} (1 - w_j(p)) \quad (3)$$

$$d(p) = \sum_i^N d_i \cdot w_i(p) \cdot \prod_j^{i-1} (1 - w_j(p)) \quad (4)$$

Where c_i represents the color of the i -th Gaussian ray and the camera ray; d_i represents the distance from the camera to the i -th Gaussian along the ray. The weight calculation is represented by $w_i(p)$, which indicates the contribution of the i -th 3D Gaussian projected onto the 2D plane at pixel p . In 3DGS, the differentiable transformation function from

3DGS (p_w , Σ_w) to 2DGS (p_c , Σ_c) is represented as follows:

$$p_c = T_{cw} p_w, \Sigma_c = J W \Sigma_w W^T J^T \quad (5)$$

Where W represents the view transformation, mainly rotation and translation, which are affine transformations; J represents the projection transformation, which is approximated as an affine transformation and then its Jacobian matrix is taken. After the above transformation, the distribution is aligned with the canvas pixels. Integrating along the third dimension yields the shading of the ellipsoid on a particular pixel. Based on the above formulas, differentiable rendering of 3DGS can be achieved.

Loss Function Cross-optimization: In each iteration, the 3DGS scene is optimized end-to-end through a weighted sum of L_1 and the Structural Similarity Index (SSIM) between the ground truth and rendered images. In our work, we designed a comprehensive loss function to optimize the model parameters. The loss function consists of the following components:

Depth Loss: Depth loss is used to measure the difference between the predicted depth map and the ground truth depth map. We use L_1 loss to calculate the depth error:

$$L_{depth} = \frac{1}{N} \sum_{i=1}^N |D_{gt}(i) - D_{pred}(i)| \quad (6)$$

Where D_{gt} is the ground truth depth map, D_{pred} is the predicted depth map, and N is the number of pixels.

RGB Loss: This loss is used to measure the difference between the predicted RGB image and the ground truth RGB image. We also use L_1 loss to calculate the RGB error:

$$L_{rgb} = \frac{1}{N} \sum_{i=1}^N |I_{gt}(i) - I_{pred}(i)| \quad (7)$$

Where I_{gt} is the ground truth RGB image, and I_{pred} is the predicted RGB image.

Multi-layer Perceptual Loss: Multi-layer perceptual loss measures image similarity by combining feature maps from multiple levels, rather than just using feature maps from a single level. This can more comprehensively capture the semantic

information and details of the image, thereby making the generated image more realistic. The formula is as follows:

$$L_p = \sum_{m=1}^M \lambda_m \|\phi_m(I_{gt}) - \phi_m(I_{pred})\|^2 \quad (8)$$

Where ϕ_m represents the feature extraction function of the m -th layer of the VGG network. λ_m is the weight of the feature map of the m -th layer. M is the total number of feature layers.

SSIM Loss: SSIM loss is used to measure the structural similarity between the predicted image and the ground truth image. We use the MS_SSIM index to calculate the loss:

$$MS_{SSIM}(x,y) = [SSIM_M(x,y)]^{a_M} \prod_{j=1}^{M-1} [SSIM_j(x,y)]^{a_j} \quad (9)$$

$$L_{ssim} = 1 - MS_{SSIM}(I_{gt}, I_{pred}) \quad (10)$$

MS-SSIM (Multi-Scale Structural Similarity) calculates SSIM at different scales and combines these SSIM values with weighted composition, thereby providing a more comprehensive measure of structural similarity between images. Compared to single-scale SSIM, MS-SSIM is better able to capture multi-level structural information in images.

$$L_{total} = \alpha L_{depth} + \beta L_{rgb} + \gamma L_{ssim} + \delta L_p \quad (11)$$

Where α , β , γ , δ are hyperparameters representing the weights of each loss term. Different loss terms may have different numerical ranges, and these weight coefficients are used to balance the contributions of each loss term. These weights can be adjusted to emphasize certain specific loss terms.

B. SLAM System

Tracking: The camera pose is represented by a rotation matrix Q and a translation vector e . A constant velocity model is used, assuming that the camera motion between consecutive frames is constant. That is, the pose of the current frame can be linearly extrapolated from the poses of the previous two frames. The rotation initialization and translation initialization use linear extrapolation of the rotation quaternions and translation vectors from the previous two frames. The formulas are as follows:

$$q_t = q_{t-1} + (q_{t-1} - q_{t-2}) \quad (12)$$

$$e_t = e_{t-1} + (e_{t-1} - e_{t-2}) \quad (13)$$

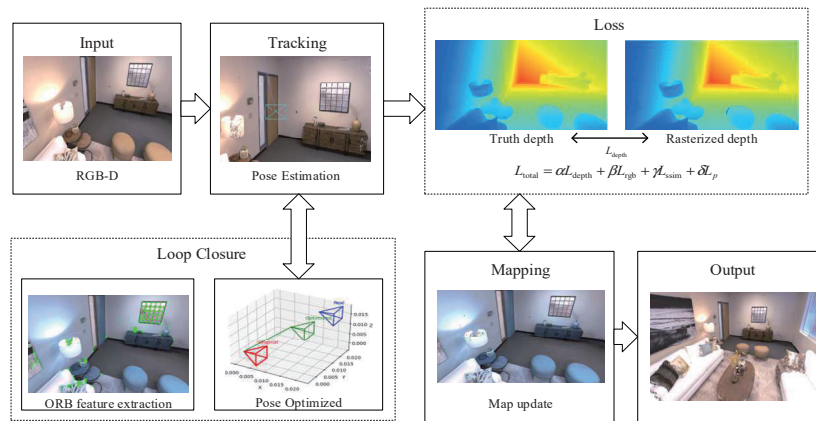


Figure 1. SLAM System Overview. After the Tracking thread receives the current RGB-D frame image as input, it estimates the camera pose and calculates the error using a comprehensive loss function. When a loop closure is detected, g2o graph optimization is used to optimize the pose parameters. The Mapping thread uses the optimized camera pose for differentiable rendering to reconstruct the scene and calculate losses

Where q_t is the rotation quaternion of the current frame, q_{t-1} and q_{t-2} are the rotation quaternions of the previous two frames respectively. e_t is the translation vector of the current frame, e_{t-1} and e_{t-2} are the translation vectors of the previous two frames respectively.

Loop Closure Detection. This stage is a crucial step in SLAM systems, used to detect whether the robot has returned to a previously visited location, thereby reducing accumulated errors. First, feature descriptors are extracted from the current frame and key frames. The code uses the ORB algorithm to extract features as shown in Fig 2 The ORB feature descriptor is a binary vector of length 32 bytes. FLANN is used for feature matching, calculating the distance between descriptors. A commonly used distance metric is the Hamming distance:

$$D=[d_1, d_2, \dots, d_{32}] \quad (14)$$

$$\text{Hamming}(D_1, D_2) = \sum_{i=1}^{32} (D_{1i} \oplus D_{2i}) \quad (15)$$

Where each d_i is a binary value.

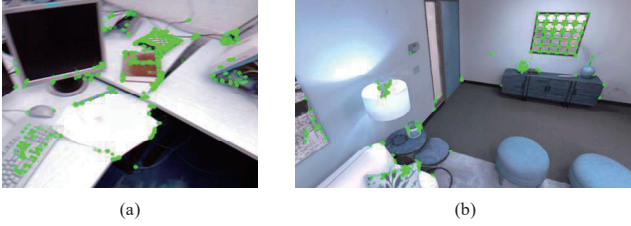


Figure 2. Figure (a) shows feature point extraction in the fr1 scene from the TUM [31] dataset, while Figure (b) shows feature point extraction in the R0 scene from the Replica [32] dataset.

We use the g2o library for pose graph optimization. First, we create an optimizer and set the solver and optimization algorithm. Then, we add camera poses to the optimizer and fix the first vertex. Next, we add the relative transformations between adjacent poses to the optimizer and set the information matrix and robust kernel function. The goal of pose graph optimization is to minimize the error of all transformations. The formula is as follows:

$$\min_{T_i} \sum_{(i,j) \in E} \|e_{ij}\|_{\Omega_{ij}}^2 \quad (16)$$

$$e_{ij} = T_{ij}^{-1} (T_i^{-1} T_j) \quad (17)$$

Where T_i represents the pose of the i -th vertex, E represents the set of all vertices. e_{ij} represents the error between the i -th vertex and the j -th vertex. The information matrix Ω_{ij} is typically a diagonal matrix used to measure the weight of the error. T_{ij} represents the relative transformation between the i -th vertex and the j -th vertex.

Mapping. The mapping stage first requires initializing Gaussian distribution parameters, including mean, rotation, scale, and opacity. Point cloud coordinates are calculated from the depth image and camera intrinsic parameters:

$$X = \left(\frac{(u-c_x) \cdot Z}{f_x}, \frac{(v-c_y) \cdot Z}{f_y}, Z \right) \quad (18)$$

In the mapping stage, the loss function is used to optimize Gaussian distribution parameters, making the map representation more accurate. Specifically, by minimizing the loss function, the mean, rotation, scale, and opacity of the Gaussian distributions are adjusted. During the mapping process, the code uses the Adam optimizer to update the parameters of the Gaussian distributions.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L \quad (19)$$

Where θ represents the parameters of the Gaussian distribution, η is the learning rate, and L is the total loss function.

IV. EXPERIMENTS

A. Experimental Setup

We implemented our method on a desktop computer equipped with an Intel i7-12700K and 24 GB NVIDIA RTX 3090 Ti. Our implementation utilizes Python programming. To ensure fair comparisons, we used experimental data from the RGB-D settings in the TUM [31] and Replica [32] datasets respectively. Our baselines include traditional methods, neural implicit methods, and state-of-the-art (SOTA) systems based on 3DGS SLAM, including iMAP [23], NICE-SLAM [24], Point-SLAM [10], GS-SLAM [28], SplaTAM [12], and MonoGS [13].

B. Evaluation

The SLAM system's performance was evaluated in tracking and mapping aspects. For tracking accuracy, we calculated the Root Mean Square Error (RMSE) of the Absolute Trajectory Error (ATE) for keyframes. Mapping quality was assessed using three standard photometric rendering metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS). Where applicable, we compared our results with those reported in relevant literature for certain datasets.

Tracking: TABLE I and TABLE II show the ATE metrics of the aforementioned baseline methods in the TUM dataset and Replica dataset, respectively. The proposed method has surpassed advanced methods based on NeRF-based SLAM

TABLE I. TRACKING PERFORMANCE COMPARISON ON TUM DATASET USING ATE (CM) ↓. IN THE TABLE ABOVE, RED, GREEN, AND PURPLE HIGHLIGHT THE FIRST, SECOND, AND THIRD BEST RESULTS RESPECTIVELY.

Class	Method	fr1	fr2	fr3	Arg
RGB-D(NeRF)	iMAP	4.90	2.00	5.80	4.23
	NICE-SLAM	4.26	6.19	6.87	5.77
	Point-SLAM	4.34	1.31	3.48	3.04
RGB-D(3DGS)	MonoGS	1.52	1.58	1.65	1.58
	SplaTAM	3.35	1.24	5.16	3.25
	GS-SLAM	3.30	1.32	6.60	3.70
	Ours	1.47	1.14	1.86	1.49

and 3DGS SLAM in the fr1 and fr2 scenes, and is only second to MonoGS among the baseline methods in the fr3 scene. This demonstrates the crucial role of loop closure detection and g2o graph optimization in improving the localization accuracy of SLAM methods and their potential in 3DGS scenes. On the Replica dataset, only the R1 and O0 scenes are inferior to SplatAM. Compared to SplatAM, which performed second-best, the average ATE metric across the five scenes in Replica improved by 24.3%, and even by 55% in the R2 scene. Although the baseline methods have already achieved remarkable millimeter-level accuracy, the proposed method remains competitive.

TABLE II. TRACKING PERFORMANCE COMPARISON ON REPLICA DATASET USING ATE (CM) ↓. IN THE TABLE ABOVE, RED, GREEN, AND PURPLE HIGHLIGHT THE FIRST, SECOND, AND THIRD BEST RESULTS RESPECTIVELY.

Method	R0	R1	R2	O0	O1	Arg
MonoGS	0.47	0.43	0.31	0.70	0.57	0.5
iMAP	3.12	2.54	2.31	1.69	1.03	2.14
NICE-SLAM	0.97	1.31	1.07	0.88	1.00	1.05
Point-SLAM	0.61	0.41	0.37	0.38	0.48	0.45
SplatAM	0.36	0.31	0.40	0.29	0.47	0.37
GS-SLAM	0.48	0.53	0.33	0.52	0.41	0.45
Ours	0.25	0.40	0.18	0.37	0.20	0.28

This is all thanks to loop closure detection, which can identify when the robot or camera returns to a previously visited location, thereby correcting accumulated localization errors during long-term motion and

maintaining accurate localization and mapping. This has improved the tracking accuracy in the SLAM system.

Rendering: TABLE III and Fig 3 show the quality of reconstructed maps for the room series and office sequence systems on the Replica dataset, compared with NICE-SLAM, Point-SLAM, MonoGS, SplatAM, and GS-SLAM. The results demonstrate competitive rendering performance even when compared to state-of-the-art NeRF and 3DGS-based SLAM systems. The proposed method achieved the best PSNR values in only two scenes, but was second only to MonoGS in the remaining scenes. For SSIM and LPIPS values, the performance was the best in all scenes except for LPIPS in O1, which was second to GS-SLAM. The average values were undoubtedly the best.

This is all due to the successful application of the designed comprehensive loss function, which specifically used multi-layer SSIM loss and added new perceptual loss, resulting in some improvement in image and structural similarity of the scenes. However, there is still room for improvement in optimizing PSNR values. It can be seen that this method is clearer and richer in texture details when reconstructing object contours, especially in furniture and floor and wall details in indoor scenes. The accuracy of depth maps has also significantly improved, showing smoother gradients and more precise object boundaries, especially in capturing the depth contours and spatial layering of indoor furniture.

TABLE III. MAPPING PERFORMANCE COMPARISON ON REPLICA DATASET. IN THE TABLE ABOVE, RED, GREEN, AND PURPLE HIGHLIGHT THE FIRST, SECOND, AND THIRD BEST RESULTS RESPECTIVELY.

Method	Metric	R0	R1	R2	O0	O1	Arg
MonoGS	PSNR[dB]↑	34.83	36.43	37.49	39.95	42.09	38.16
	SSIM↑	0.954	0.959	0.965	0.971	0.977	0.965
	LPIPS↓	0.068	0.076	0.075	0.072	0.055	0.069
NICE-SLAM	PSNR[dB]↑	22.12	22.47	24.52	29.07	30.34	25.7
	SSIM↑	0.689	0.757	0.814	0.874	0.886	0.804
	LPIPS↓	0.33	0.271	0.208	0.229	0.181	0.244
SplatAM	PSNR[dB]↑	32.86	33.89	35.25	38.26	31.97	34.45
	SSIM↑	0.980	0.970	0.980	0.980	0.970	0.976
	LPIPS↓	0.07	0.10	0.08	0.09	0.10	0.09
GS-SLAM	PSNR[dB]↑	31.56	32.86	32.59	38.70	41.17	35.38
	SSIM↑	0.968	0.973	0.971	0.986	0.993	0.978
	LPIPS↓	0.094	0.075	0.093	0.050	0.033	0.069
Ours	PSNR[dB]↑	33.73	35.55	37.59	41.06	41.85	37.96
	SSIM↑	0.980	0.982	0.988	0.990	0.994	0.987
	LPIPS↓	0.053	0.054	0.052	0.048	0.047	0.051

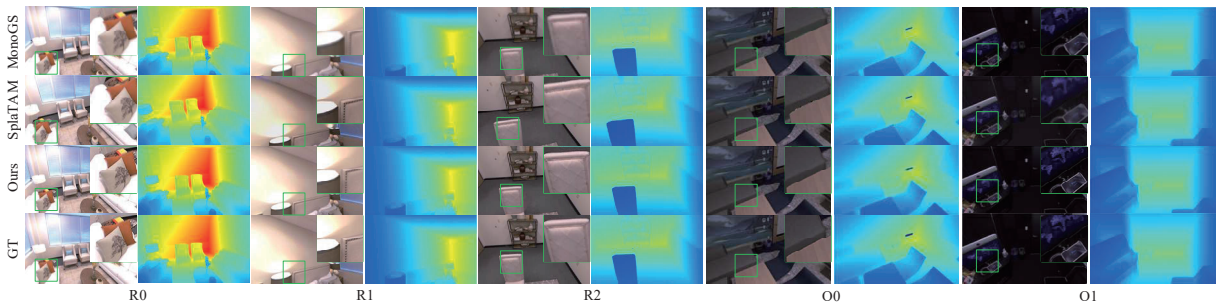


Figure 3. In the Replica dataset, reconstruction experiments were conducted on 3 room scene sequences. The results were compared with MonoGS, SplatAM, and the depth maps of the real environment.

V. CONCLUSION

This article proposes a 3DGS SLAM system based on closed-loop detection and comprehensive loss function. By utilizing the prior information provided by loop closure adjustment, high-quality tracking and reconstruction of the scene have been achieved, global loop closure has been achieved, accumulated errors have been corrected, and positioning accuracy has been improved. High quality texture detail representation was achieved using multi-layer perceptions in the TUM and Replica datasets.

The method proposed in this article performs well in tracking, rendering, and mapping, demonstrating its potential in practical applications. For example, autonomous vehicles navigating in complex urban environments require real-time construction of high-precision maps and positioning. VR systems can generate high fidelity virtual scenes in real time and ensure the accuracy of user positioning and interaction in the virtual environment through optimized SLAM algorithms. Future research can further optimize loop closure detection and correction methods to cope with larger scale scenarios.

REFERENCES

- [1] B Mildenhall, P P Srinivasan, M Tancik, et al. Nerf: Representing scenes as neural radiance fields for view synthesis[J]. *Communications of the ACM*, 2021, 65(1): 99-106.
- [2] A Rosinol, J J Leonard, L Carlone. Nerf-slam: Real-time dense monocular slam with neural radiance fields[C]//2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2023: 3437-3444.
- [3] B Kerbl, G Kopanas, T Leimkühler, et al. 3D Gaussian Splatting for Real-Time Radiance Field Rendering[J]. *ACM Trans. Graph.*, 2023, 42(4): 139:1-139:14.
- [4] R A Newcombe, S J Lovegrove, A J Davison. DTAM: Dense tracking and mapping in real-time[C]//2011 international conference on computer vision. IEEE, 2011: 2320-2327.
- [5] E Rublee, V Rabaud, K Konolige, et al. ORB: An efficient alternative to SIFT or SURF[C]//2011 International conference on computer vision. IEEE, 2011: 2564-2571.
- [6] M Nießner, M Zollhöfer, S Izadi, et al. Real-time 3D reconstruction at scale using voxel hashing[J]. *ACM Transactions on Graphics (ToG)*, 2013, 32(6): 1-11.
- [7] C Sun, M Sun, H T Chen. Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2022: 5459-5469.
- [8] X Yang, H Li, H Zhai, et al. Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation[C]//2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR). IEEE, 2022: 499-507.
- [9] Y Zhang, F Tosi, S Mattoccia, et al. Go-slam: Global optimization for consistent 3d instant reconstruction[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 3727-3737.
- [10] E Sandström, Y Li, L Van Gool, et al. Point-slam: Dense neural point cloud-based slam[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2023: 18433-18444.
- [11] H Huang, L Li, H Cheng, et al. Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular Stereo and RGB-D Cameras[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 21584-21593.
- [12] N Keetha, J Karhade, K M Jatavallabhula, et al. SplatTAM: Splat Track & Map 3D Gaussians for Dense RGB-D SLAM[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 21357-21366.
- [13] H Matsuki, R Murai, P H J Kelly, et al. Gaussian splatting slam[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 18039-18048.
- [14] M Muja, D G Lowe. Flann, fast library for approximate nearest neighbors[C]//International Conference on Computer Vision Theory and Applications (VISAPP'09). Setúbal, Portugal: INSTICC Press, 2009, 3: 1-21.
- [15] M Aumüller, E Bernhardsson, A Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms[J]. *Information Systems*, 2020, 87: 101374.
- [16] C Campos, R Elvira, J J G Rodríguez, et al. Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam[J]. *IEEE Transactions on Robotics*, 2021, 37(6): 1874-1890.
- [17] V Ila, L Polok, M Solony, et al. SLAM++-A highly efficient and temporally scalable incremental SLAM framework[J]. *The International Journal of Robotics Research*, 2017, 36(2): 210-230.
- [18] M Ramezani, Y Wang, M Camurri, et al. The newer college dataset: Handheld lidar, inertial and vision with ground truth[C]//2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2020: 4353-4360.
- [19] R Mur-Artal, J D Tardós. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras[J]. *IEEE transactions on robotics*, 2017, 33(5): 1255-1262.
- [20] X Chen, T Labe, A Milioto, et al. OverlapNet: Loop closing for LiDAR-based SLAM[J]. *arXiv preprint arXiv:2105.11344*, 2021.
- [21] M Bloesch, J Czarnowski, R Clark, et al. Codeslam—learning a compact, optimisable representation for dense visual slam[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 2560-2568.
- [22] P E Sarlin, D DeTone, T Malisiewicz, et al. Superglue: Learning feature matching with graph neural networks[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2020: 4938-4947.
- [23] E Sucar, S Liu, J Ortiz, et al. imap: Implicit mapping and positioning in real-time[C]//Proceedings of the IEEE/CVF international conference on computer vision. 2021: 6229-6238.
- [24] Z Zhu, S Peng, V Larsson, et al. Nice-slam: Neural implicit scalable encoding for slam[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2022: 12786-12796.
- [25] H Wang, J Wang, L Agapito. Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 13293-13302.
- [26] I O Tolstikhin, N Houlsby, A Kolesnikov, et al. Mlp-mixer: An all-mlp architecture for vision[J]. *Advances in neural information processing systems*, 2021, 34: 24261-24272.
- [27] M M Johari, C Carta, F Fleuret. Eslam: Efficient dense slam system based on hybrid representation of signed distance fields[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 17408-17419.
- [28] C Yan, D Qu, D Xu, et al. Gs-slam: Dense visual slam with 3d gaussian splatting[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 19595-19604.
- [29] S Ha, J Yeon, H Yu. Rgb-d gs-icp slam[J]. *arXiv preprint arXiv:2403.12550*, 2024.
- [30] V Yugay, Y Li, T Gevers, et al. Gaussian-slam: Photo-realistic dense slam with gaussian splatting[J]. *arXiv preprint arXiv:2312.10070*, 2023.
- [31] J Sturm, N Engelhard, F Endres, et al. A benchmark for the evaluation of RGB-D SLAM systems[C]//2012 IEEE/RSJ international conference on intelligent robots and systems. IEEE, 2012: 573-580.
- [32] J Straub, T Whelan, L Ma, et al. The Replica dataset: A digital replica of indoor spaces[J]. *arXiv preprint arXiv:1906.05797*, 2019.